Reporte

Técnico del Proyecto

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ
Facultad de Ingeniería
Área de Ciencias de la Computación

Interfaces Gráficas con Aplicaciones

Juego UNO Multijugador

Integrantes :

- Ángel Daniel Cruz García
- Javier Alejandro Patiño Medrano
- Daniel Ramírez Khun
- Sofia Nazllely Ruiz Cárdenas

Fecha: 6 de noviembre de 2026

Repositorio: github.com/danielrmzkuhn1/Uno

CONTENIDO

1. Presentación y objetivo del proyecto
2. Funcionamiento y estructura del sistema
3. División del trabajo
4. Modelo de datos y persistencia
5. Desarrollo detallado de la segunda entrega
6. Trabajo colaborativo con Git y GitHub
7. Dificultades, decisiones y mejoras
8. Resultados, pruebas y conclusiones
Anexo A. Script SQL del modelo relacional
Anexo B. Capturas de la interfaz y evolución del juego
Anexo C. Pruebas y fragmentos de implementación
Anexo D. Procedimiento de integración y verificación



1. PRESENTACIÓN Y OBJETIVO DEL PROYECTO

El proyecto consistió en implementar una versión digital del juego UNO con una interfaz gráfica en C# y un sistema de almacenamiento relacional. En la primera etapa, el juego se planteó para la interacción local en una misma computadora. En la segunda entrega se amplió a partidas multijugador, donde distintas computadoras conectadas a la misma red Wi-Fi se comunican con un servidor TCP desarrollado en Python.

El resultado integra las reglas de UNO, la visualización de cartas y turnos, el manejo de salas, el envío de acciones al servidor y el registro de jugadores, partidas y movimientos en MySQL. La ampliación requirió reorganizar la comunicación y modificar varias pantallas de C#; también se añadió una relación explícita de participantes por partida en SQL.

1.1 OBJETIVOS ESPECÍFICOS

Implementar el reparto, los turnos, las cartas especiales, la declaración UNO y la condición de victoria.

Permitir que de dos a cuatro jugadores creen o se unan a una sala desde computadoras distintas.

Sincronizar el estado de la partida sin mostrar a otros jugadores las cartas privadas de cada mano.

Registrar en MySQL los jugadores, la fecha, el ganador, los participantes y los eventos realizados.

Organizar el desarrollo con ramas, commits y revisiones de integración en GitHub.

1.2 CAMBIO DE ALCANCE ENTRE ENTREGAS

Primera etapa	Segunda etapa
Juego local y mesa gráfica en C#.	Juego en red local con clientes C# y servidor Python TCP.
Lógica de UNO implementada en clases C#.	Reglas y estado compartido administrados por motor.py en el servidor.
Modelo SQL de jugadores, partidas e historial.	Se agregó partida_jugadores para identificar participantes y derivar resultados individuales.

2. FUNCIONAMIENTO Y ESTRUCTURA DEL SISTEMA

2.1 ARQUITECTURA CLIENTE SERVIDOR

La interfaz Windows Forms envía solicitudes mediante la clase ApiService.cs. Aunque conserva ese nombre, ya no utiliza peticiones HTTP: abre una conexión TCP persistente con una dirección IP y un puerto. Cada solicitud se representa mediante un objeto JSON codificado en UTF-8 y separado por un salto de línea.



El archivo servidor_sockets.py recibe conexiones, interpreta mensajes y publica cambios a los clientes de la sala. Llama a api.py para las operaciones con usuarios, salas y persistencia, y a motor.py para las reglas del juego. MySQL conserva información entre sesiones. Esta separación permite que la interfaz consulte un estado autorizado sin modificar directamente las reglas ni las consultas SQL.

2.2 COMPONENTES Y ARCHIVOS PRINCIPALES

Componente	Archivos	Responsabilidad
Cliente C#	Form1.cs, ApiService.cs	Muestra el menú y la mesa; crea y une salas; envía acciones y actualiza la interfaz al recibir estados.
Lógica local previa	Logica/Carta.cs, Mazo.cs, Juego.cs, Jugador.cs, Reglas.cs	Representa el diseño orientado a objetos de la primera etapa.
Servidor TCP	servidor/servidor_sockets.py	Acepta conexiones y distribuye estados mediante mensajes JSON.

2.3 FLUJO DE UNA PARTIDA MULTIJUGADOR

1. Un participante escribe su nombre y la dirección del servidor, y crea una sala. El servidor genera un código y un token de sesión.
2. Otros participantes escriben el código de sala y se incorporan desde sus computadoras. La capacidad de la sala es de dos a cuatro jugadores.
3. El creador inicia la partida. El motor prepara las 108 cartas, reparte siete a cada participante y asigna el turno inicial.
4. Cada acción de jugar, robar o declarar UNO se envía al servidor junto con la versión del estado. El servidor la valida antes de modificar el juego.
5. El servidor guarda los movimientos y publica el estado actualizado. Cada cliente recibe su propia mano y solo el número de cartas de los rivales.
6. Cuando alguien se queda sin cartas, se registra el ganador. El creador puede preparar otra partida conservando la sala y las identidades.

2.4 REGLAS, CONSISTENCIA Y PRIVACIDAD

El motor admite cartas numéricas, Salto, Reversa, +2, Comodín y +4. Una jugada es válida si coincide con el color, número o tipo correspondiente, o si utiliza un comodín permitido. Al quedarse con una carta se abre un intervalo de cinco segundos para declarar UNO; si expira, el motor aplica un castigo de dos cartas.

Para evitar inconsistencias de red, api.py compara la versión enviada con la versión almacenada y mantiene identificadores de solicitud para reconocer reintentos. El servidor autentica por token y devuelve a cada jugador únicamente su mano. Así se protege la información privada mientras las cantidades de cartas y el turno permanecen visibles para todos.

2.5 PROTOCOLO DE COMUNICACIÓN ENTRE C# Y PYTHON

En la segunda etapa se adoptó una conexión TCP persistente en lugar de depender de llamadas HTTP independientes. El cliente conserva un socket abierto durante su participación en la sala. Esta decisión permite recibir notificaciones cuando otra persona se une, realiza una jugada o termina la partida, sin que cada equipo tenga que consultar manualmente al servidor después de cada movimiento.

El protocolo transmite un objeto JSON por línea. La clase ApiService.cs agrega un identificador id único a cada solicitud y espera una respuesta con ese mismo identificador. El servidor responde con tipo respuesta, un indicador ok y, cuando corresponde, el estado actualizado o un mensaje de error. Además puede enviar mensajes independientes de tipo estado para avisar cambios a todos los clientes conectados.

Tipo de mensaje	Finalidad en el juego
crear / entrar	Crea una sala o incorpora a un jugador mediante el código compartido.
consultar	Solicita una vista actualizada del estado para el usuario autenticado.
accion	Envía iniciar, jugar, robar o uno, junto con versión y datos necesarios.
revancha	Solicita preparar otra partida una vez que la anterior terminó.
cancelar / salir	Permite al creador cancelar antes de iniciar o a un invitado retirarse.
reanudar	Permite al servidor reconocer una sesión previa usando código y token.
estado	Notificación del servidor con la vista autorizada de cada destinatario.

Ejemplo ilustrativo de una solicitud de robo de carta, con valores abreviados para explicar la estructura del intercambio:

{"id":"sol-001","tipo":"accion",
"datos":{"accion":"robar","version":12,"solicitud":"mov-001"}}

La propiedad id relaciona la respuesta TCP con la solicitud que la originó. La propiedad solicitud, dentro de datos, tiene otro propósito: evita aplicar dos veces una misma jugada si la petición se retransmite. La versión indica sobre qué estado intentó actuar el jugador. Los identificadores del ejemplo son ilustrativos; la implementación real genera valores únicos.

2.6 ACTUALIZACIÓN DE CLIENTES Y MANEJO DE CONCURRENCIA

servidor_sockets.py utiliza asyncio para atender conexiones simultáneas. Mantiene un conjunto de clientes activos, reconoce a qué sala pertenece cada uno y conserva un candado asíncrono durante la operación de lectura, validación y publicación. Así se coordina el acceso al estado compartido cuando dos computadoras realizan solicitudes muy cercanas en el tiempo.

Después de procesar una petición que corresponde a una sala, el servidor llama a publicar. Esta rutina consulta el estado con el token de cada cliente por separado; por ello no envía un mismo objeto con todas las manos. Un participante recibe su propia lista de cartas, la carta superior, el color activo, la dirección del juego, el turno y el número de cartas de las demás personas. El cliente actualiza su vista mediante el evento EstadoRecibido.

En C#, las respuestas esperadas se administran mediante una colección de solicitudes pendientes. ApiService.cs utiliza SemaphoreSlim para evitar escrituras simultáneas sobre el mismo flujo de salida. El método Leer distingue las respuestas individuales de las notificaciones de estado. Form1.cs emplea BeginInvoke para dibujar los cambios en el hilo de la interfaz de Windows Forms.

El servidor acepta las conexiones en el puerto 8000 de forma predeterminada, configurable con UNO_PORT. Escucha sobre 0.0.0.0 para recibir conexiones de la red; los clientes deben escribir la IP local del equipo anfitrión y el puerto correcto. El valor 127.0.0.1 que aparece inicialmente en el formulario corresponde a pruebas donde el cliente y el servidor están en la misma computadora.

2.7 VERSIONES, REINTENTOS Y RECUPERACIÓN DE ERRORES

Cada estado de la partida tiene un contador version. Cuando el usuario intenta jugar, Form1.cs adjunta la versión que estaba mostrando. Si otro participante ya provocó un cambio, api.py detecta que la solicitud se construyó a partir de información anterior y devuelve un error de conflicto. La interfaz puede consultar nuevamente el servidor antes de permitir otro intento.

El servidor también conserva, dentro del estado de la sala, los identificadores recientes de solicitudes procesadas. Si una acción llega repetida con el mismo identificador lógico, no se vuelve a aplicar el efecto. Esta protección importa especialmente para robos y castigos: procesar dos veces una misma solicitud alteraría injustamente la cantidad de cartas.

En el cliente se contemplan fallas de conexión y límites de espera. La conexión inicial dispone de un tiempo máximo de ocho segundos y las solicitudes de diez segundos; si se supera alguno, se informa al usuario. La interfaz deshabilita temporalmente controles mientras procesa una acción, evitando clics repetidos, y muestra un aviso cuando es necesario volver a consultar el estado. Esta recuperación no equivale a una reconexión automática completa: ante un cierre definitivo del socket, el formulario informa que debe reiniciarse el juego para abrir otra sala.

3. División del trabajo

El equipo mantuvo las cuatro responsabilidades principales definidas al inicio. Durante la segunda etapa, la integración exigió que cada módulo se ajustara a la ejecución por red y a las reglas de almacenamiento comunes.

Integrante	Área asignada	Avances documentados
Ángel	Lógica del juego	Clases originales de UNO; corrección del uso de nombres entre partidas; registro de participantes y revancha iniciada por el creador.
Sofía	Interfaz gráfica	Menú, mesa, imágenes, controles de cartas, botón UNO y temporizador; adaptación visual para integrar las operaciones del servidor.
Daniel	Comunicación con C# y servidor	Cliente C# mediante ApiService, conexión TCP, ajustes del menú y botones de salir o cancelar la sala.
Alex	Base de datos y conexión	Diseño de tablas SQL, participación de jugadores por partida, integración del servidor y pruebas de multijugador mediante sockets.

3.1 ÁNGEL: LÓGICA Y CONTROL DE PARTIDAS

En la primera etapa desarrolló la estructura orientada a objetos de cartas, jugadores, mazo y partida. En la integración posterior se reflejan cambios en la identidad de participantes y en la continuidad entre partidas. La función obtener_jugador de api.py reutiliza el identificador de un nombre ya registrado, mientras que la función revancha conserva las identidades de la sala y prepara otra partida después de que existe un ganador.

Los commits del 4 de noviembre documentan la corrección de nombres reutilizados, el seguimiento de participantes en cada partida y las revanchas controladas por el creador. Este trabajo es relevante para la consistencia entre el historial SQL y las partidas visibles para los usuarios.

3.2 SOFIA: INTERFAZ GRÁFICA Y EXPERIENCIA DEL USUARIO

Durante la primera entrega se diseñaron el menú principal, el fondo visual, la mesa con las cartas y los controles de interacción. Se representaron los cuatro espacios de jugadores, la carta superior, el sentido del turno, el mazo y las acciones disponibles. También se incorporó el botón UNO con temporizador y la visualización de cartas rivales.

En la versión conectada, Form1.cs muestra la dirección del servidor, el nombre y el código de sala. La interfaz actualiza los elementos según la fase de espera, partida iniciada o partida terminada; por ejemplo, solo muestra iniciar partida al creador y habilita el juego cuando corresponde el turno.

3.3 DANIEL: COMUNICACIÓN E INTEGRACIÓN DE C#

ApiService.cs emplea TcpClient, StreamReader y StreamWriter para mantener la conexión con el servidor. Envía mensajes JSON de tipo crear, entrar, consultar, accion, revancha, cancelar y salir, y escucha actualizaciones del estado sin necesidad de pedirlo constantemente. También contempla errores de conexión y tiempos máximos de espera.

La segunda etapa añadió controles para abandonar una sala antes del inicio y para cancelar la sala si el usuario es su creador. Se ajustaron los mensajes y la visibilidad de los botones para que aparezcan en el momento apropiado.

3.4 ALEX: PERSISTENCIA Y SOPORTE AL MULTIJUGADOR

El esquema SQL inicial guardaba jugadores, partidas y eventos. La tabla partida_jugadores amplió el modelo mediante una llave primaria compuesta, de modo que una misma persona pueda participar en distintas partidas y cada partida tenga varios participantes. La inserción se realiza cuando comienza la partida, a través de registrar_participantes en api.py.

El historial del repositorio también incluye pruebas del servidor y el agregado de multijugador mediante sockets TCP. La integración exige crear en MySQL todas las tablas usadas por el servidor antes de ejecutar una partida.

4. MODELO DE DATOS Y PERSISTENCIA

4.1 EVOLUCIÓN DEL ESQUEMA RELACIONAL

La primera versión contenía tres tablas: jugadores, partidas e historial_partida. Ese diseño registraba al ganador de cada partida, pero no permitía conocer de forma directa a todos los participantes, porque un jugador podía aparecer en una partida sin haber registrado todavía un movimiento. En la segunda etapa se incorporó partida_jugadores para representar correctamente la participación.

La nueva tabla es una entidad asociativa entre jugadores y partidas, con id_partida e id_jugador como llave primaria conjunta. La combinación impide repetir dos veces al mismo jugador dentro de una misma partida. Ambas columnas son llaves foráneas, por lo que sus valores deben existir en las tablas originales.



Modelo relacional correspondiente a las cuatro tablas declaradas en el script uno_db.sql.

4.2 DESCRIPCIÓN DE LAS TABLAS

Tabla	Llave principal	Información y función
jugadores	id_jugador	Nombre e identificador persistente de una persona.
partidas	id_partida	Fecha y llave foránea id_ganador, que puede permanecer NULL mientras no termine.
partida_jugadores	(id_partida, id_jugador)	Asociación de participantes; ambas columnas son PK y FK.
historial_partida	id_evento	Partida, actor, orden de evento, acción y datos de la carta.

4.3 RELACIONES E INTEGRIDAD REFERENCIAL

jugadores 1 a N partidas: un jugador puede ganar varias partidas; cada partida registra a lo sumo un ganador.
jugadores 1 a N partida_jugadores y partidas 1 a N partida_jugadores: en conjunto resuelven la relación muchos a muchos.
jugadores 1 a N historial_partida y partidas 1 a N historial_partida: cada evento está asociado a un actor y una partida existentes.
La columna turno de historial_partida funciona en la implementación como un consecutivo de eventos de cada partida: api.py obtiene el valor máximo existente y aumenta uno por cada nuevo registro. Las columnas color_carta y valor_carta aceptan NULL para acciones que no corresponden a una carta específica.

4.4. REGISTRO DE PARTICIPANTES Y RESULTADOS

Al recibir la acción iniciar, api.py llama a registrar_participantes y añade una fila por jugador de la sala a partida_jugadores. Cuando se declara un ganador, actualizar partidas.id_ganador permite obtener el resultado de cada participante: victoria si coincide con ese identificador y derrota para los demás, una vez finalizada la partida.
La consulta de ejemplo incluida en el anexo C muestra cómo recuperar los resultados sin añadir una columna redundante de derrotas. La condición de que el ganador pertenezca a la lista de participantes se valida dentro del juego; el esquema SQL mostrado no la impone por sí mismo.

4.5 TABLA AUXILIAR UTILIZADA POR EL SERVIDOR

El código de api.py también utiliza salas_wifi, donde consulta o actualiza los campos codigo, id_partida y estado. Esta tabla conserva el identificador de sala y el estado serializado del juego mientras funciona la sesión. El equipo confirmó su creación directamente en MySQL; sin embargo, su sentencia CREATE TABLE todavía no aparece en el archivo uno_db.sql suministrado. Para una entrega reproducible debe anexarse esa definición exacta y actualizar el diagrama final.

4.6 RECORRIDO DE LOS DATOS DURANTE UNA PARTIDA

Para comprender la utilidad de las tablas, es conveniente seguir una partida desde la creación de la sala hasta el registro del ganador. La información no se almacena toda en una sola tabla: cada relación tiene una función distinta y las operaciones de api.py mantienen sincronizados los registros persistentes con el estado del motor.

Momento	Operación en el servidor
Creación de sala	Se valida el nombre, se identifica o registra al creador y se prepara un estado en fase de espera.
Unión de invitados	Se verifica que la sala admita más personas y que el nombre no esté repetido dentro de ella.
Inicio de partida	El creador inicia con dos a cuatro personas; se registran todos los participantes.
Acciones de juego	Se aplica la regla y se vuelcan los eventos pendientes de la operación.
Victoria	El motor determina quién agotó sus cartas y registra su identificador como ganador.
Nueva partida	Se crea otro identificador de partida y se conserva la identidad de los jugadores en la misma sala.

El método guardar de api.py recoge los eventos generados por motor.py, calcula el siguiente consecutivo de la partida y realiza los INSERT correspondientes en historial_partida. Si el motor ya declaró un ganador, actualiza partidas.id_ganador. Finalmente sobrescribe el estado JSON de la sala. Las operaciones se realizan dentro de una transacción: si ocurre una excepción durante el proceso, la conexión a MySQL solicita una reversión en lugar de dejar una escritura parcial.

En una ejecución normal, una partida iniciada debe tener entre dos y cuatro filas en partida_jugadores. Esa relación permite contar participaciones aunque una persona no haya alcanzado a realizar ninguna acción. El historial, por su parte, conserva las acciones registradas y los datos de carta cuando corresponden. Por eso las dos tablas no se sustituyen entre sí.

4.7 Utilidad del modelo para consultas posteriores

El nuevo modelo permite elaborar información que la primera versión no podía calcular con seguridad. Para obtener las partidas disputadas por un jugador, se consulta partida_jugadores; para conocer sus victorias se compara su id con id_ganador; y para calcular derrotas se consideran solo partidas terminadas en las que participó, pero no ganó. El resultado se puede construir con JOIN entre jugadores, partida_jugadores y partidas.
También es posible consultar una secuencia cronológica de acciones utilizando id_partida y el consecutivo turno de historial_partida. Esto puede servir para revisar por qué un jugador recibió un castigo o qué movimientos se registraron antes de finalizar.
Aunque el proyecto no incluye necesariamente una pantalla de estadísticas, la estructura deja preparada la información para realizar estas consultas en MySQL.
Un aspecto importante de integridad es que las llaves foráneas garantizan la existencia de los identificadores referenciados, pero no verifican todas las reglas del juego. Por ejemplo, el esquema por sí solo no garantiza que el ganador haya sido uno de los participantes de la partida; esa condición depende de la lógica del servidor. Diferenciar las restricciones de SQL de las reglas de la aplicación evita atribuir a la base de datos funciones que no realiza.

5. Desarrollo detallado de la segunda entrega

La primera entrega permitió presentar una versión local con elementos visuales y reglas implementadas. Durante la segunda etapa se trabajó sobre esa base para que las partidas dejaran de depender de una sola computadora. Fue necesario agregar archivos Python, cambiar el mecanismo de comunicación de C#, ampliar la persistencia y adaptar las pantallas para cubrir situaciones que no existían en el modo local, como la espera de invitados, la salida de una sala y la continuidad después de una victoria.

5.1 INCORPORACIÓN DEL SERVIDOR PYTHON

Los archivos nuevos del directorio servidor distribuyen las responsabilidades del multijugador. motor.py contiene el estado y las reglas de UNO sin depender directamente de MySQL ni de la interfaz. api.py coordina la creación de salas, la autenticación por token, la recuperación y escritura de información; también define las operaciones que pueden solicitar los clientes. servidor_sockets.py abre el puerto de escucha y transporta las peticiones y respuestas.

Esta división hace que las reglas se ejecuten en un punto común. Si cada cliente decidiera por su cuenta si una carta es válida o qué jugador sigue, dos equipos podrían mostrar resultados incompatibles. En la arquitectura desarrollada, C# solicita acciones, Python decide si están permitidas y luego devuelve una vista del estado resultante. Las computadoras muestran la misma partida, pero cada una recibe datos privados de su propio jugador.

Además se añadieron pruebas separadas del motor, los servicios y los sockets. Los archivos test_motor.py, test_api.py y test_sockets.py representan escenarios que son difíciles de revisar únicamente mediante clics en la interfaz, como peticiones duplicadas, mensajes TCP fragmentados, turnos inválidos o expiración del temporizador UNO.

5.2 CAMBIOS EN C# PARA CONECTARSE POR RED

La clase ApiService.cs cambió su función principal: en lugar de intercambiar peticiones HTTP, utiliza TcpClient, StreamReader y StreamWriter para mantener una comunicación bidireccional. Al entrar a una sala, el cliente valida la dirección escrita, intenta conectarse y comienza una tarea de lectura que permanece activa mientras dure la sesión.

El código asocia cada petición enviada con su respuesta mediante un identificador. Esta decisión permite que el servidor envíe una notificación mientras el cliente todavía está esperando la contestación de una acción anterior. La recepción continua del estado no bloquea la interfaz; cuando llega una actualización, Form1.cs llama a los métodos necesarios para redibujar las cartas, el turno, la sala y los mensajes.

Se conservó el nombre ApiService por compatibilidad con la estructura anterior, aunque el protocolo utilizado ahora es TCP. La comunicación actual se realiza mediante un socket persistente y no mediante una API HTTP tradicional.

5.3 PANTALLA DE CONEXIÓN Y ADMINISTRACIÓN DE SALAS

El menú pasó de una entrada principalmente visual a una pantalla de conexión. Ahora solicita la dirección IP y el puerto del servidor, el nombre del participante y un código de sala para quienes desean unirse. Un jugador puede crear una sala nueva; los demás introducen el código que comparte el anfitrión. El servidor genera un identificador de sala y asigna a cada participante un token de sesión.

El menú pasó de una entrada principalmente visual a una pantalla de conexión. Ahora solicita la dirección IP y el puerto del servidor, el nombre del participante y un código de sala para quienes desean unirse. Un jugador puede crear una sala nueva; los demás introducen el código que comparte el anfitrión. El servidor genera un identificador de sala y asigna a cada participante un token de sesión.

Fase o condición	Qué muestra o permite la interfaz
Sin conexión	Captura de IP:puerto, nombre, código y opciones de crear o unirse.
Sala en espera, creador	Código, integrantes y botones de iniciar o cancelar; iniciar exige al menos dos jugadores.
Sala en espera, invitado	Lista de integrantes, mensaje de espera y opción de salir antes del inicio.
Partida en curso	Carta superior, color, turno, mazo, cartas propias y controles habilitados según el estado.
Ventana UNO	Botón ¡UNO! para el jugador al que corresponde declarar antes del vencimiento.
Partida terminada	Nombre del ganador y opción de nueva partida para el creador; los demás esperan su decisión.
Sala cancelada	Aviso de cancelación y botón para volver al menú.

La captura de la segunda entrega muestra la sala Wi-Fi con estos nuevos campos y un código para compartir. Esa imagen permite identificar la modificación del menú, mientras que las capturas de la primera etapa sirven como comparación con el diseño local anterior.

5.4 Inicio de partida y sincronización de turnos

Cuando el creador presiona Iniciar partida, el cliente envía una acción iniciar. El servidor valida su identidad y comprueba que la sala siga esperando y que haya al menos dos jugadores. motor.py construye un mazo de 108 cartas, lo mezcla, reparte siete por participante y elige una carta numérica inicial para el descarte. api.py registra a las personas de la sala en partida_jugadores y guarda el evento de inicio.

Las acciones posteriores se validan siempre contra el turno y las reglas comunes. Si una persona intenta jugar una carta incompatible, seleccionar un índice que no existe en su mano o actuar fuera de turno, el motor no debe modificar el estado y devuelve un error. Las cartas Salto, Reversa, +2 y +4 modifican el orden o la cantidad de cartas según la implementación; la Reversa también recibe tratamiento particular cuando solo hay dos jugadores.

Tras una jugada válida, el servidor publica los estados individuales. El cliente que acaba de jugar ve la carta retirada de su mano; las demás computadoras actualizan la carta superior y la cantidad de cartas de ese jugador, sin conocer su mano completa. El flujo evita que una interfaz tenga que calcular por su cuenta el siguiente turno.

5.5 Implementación de UNO y temporizador

La primera versión ya incluía un botón UNO con temporizador en la mesa. Para el multijugador, el plazo pasó a depender del estado compartido del motor. Cuando un jugador queda con una carta, se registra a quién corresponde declarar UNO y el instante límite calculado a partir del reloj del servidor.

servidor_sockets.py programa una tarea asíncrona vinculada a esa sala. Si el plazo de cinco segundos termina sin declaración, la tarea ejecuta la penalización de dos cartas y notifica el nuevo estado; no necesita esperar otra acción del usuario. La declaración correcta cancela la condición pendiente. La vista que recibe C# incluye uno_jugador y uno_segundos para que el botón aparezca solamente cuando corresponde.

Este cambio era importante porque el contador visual de una sola computadora no basta para arbitrar una partida entre varios equipos. El servidor conserva la decisión final sobre la expiración, mientras cada cliente representa el resultado en pantalla.

5.6 Abandonar y cancelar salas

En una partida por red deben contemplarse los casos en los que alguien decide no jugar después de entrar. Se agregaron dos acciones distintas. Salir se utiliza cuando un invitado abandona una sala que todavía no ha iniciado; se elimina de la lista de participantes de esa sala y se informa el cambio a quienes permanecen conectados. Cancelar corresponde al creador y marca la sala como cancelada para que las demás personas reciban un aviso.

La interfaz utiliza botones diferentes y confirma la cancelación antes de enviarla. Form1.cs también oculta los controles hasta que existe una sesión válida, y muestra la opción de regresar al menú cuando la sala ha sido cancelada. En el servidor se valida que estas operaciones ocurran durante la fase de espera. El creador no utiliza Salir: debe cancelar la sala. Estas restricciones evitan que un abandono imprevisto modifique una partida que ya está en curso.
Al completar la salida o cancelación, VolverAlMenu restablece los campos y controles de la interfaz, libera la conexión anterior y permite crear otra sala. Así se evita que el formulario conserve datos visuales de una sesión que ya terminó.

5.7 GANADOR Y NUEVA PARTIDA

La partida finaliza cuando un jugador descarta su última carta, momento en el que motor.py cierra el juego, api.py registra al ganador en la base de datos y la interfaz anuncia al vencedor bloqueando las acciones habituales. Para agilizar la continuidad del grupo sin tener que crear una sala desde cero, se incorporó la opción de "Nueva partida" exclusiva para el anfitrión (con confirmación previa). Al validarse que la ronda anterior concluyó, el servidor genera un nuevo identificador y reinicia el mazo, las manos, los turnos y el ganador, manteniendo intactos los datos y tokens de los participantes en la sala para que cada partida conserve su propio historial de forma independiente.

5.8 PREPARACIÓN PARA JUGAR ENTRE VARIAS COMPUTADORAS

Para probar el sistema en red local, es indispensable configurar correctamente la conectividad hacia el equipo anfitrión y actualizar manualmente la estructura de la base de datos. En el entorno de red, el servidor debe ejecutar Python y MySQL mientras los clientes se conectan mediante su dirección IP local y puerto correspondiente —reemplazando 127.0.0.1, que solo apunta al propio equipo— y asegurando que los firewalls permitan el tráfico TCP necesario. Asimismo, debido a que comandos como git pull actualizan el código pero no modifican la base de datos por sí solos, cada integrante debe ejecutar manualmente la sentencia CREATE TABLE para incorporar la tabla partida_jugadores y mantener sincronizado el esquema SQL con la versión más reciente del repositorio.

6. TRABAJO COLABORATIVO CON GIT Y GITHUB

6.1 ORGANIZACIÓN DEL REPOSITORIO

Se mantuvo main como rama de integración y se utilizaron ramas para separar la lógica, la interfaz, la comunicación y la base de datos. Los merges permitieron reunir los módulos y corregir incompatibilidades. El repositorio se conserva en github.com/danielrmzkuhn1/Uno. Las contribuciones se identifican en GitHub mediante las cuentas angelddev (Ángel), sofii15 (Sofía), danielrmzkuhn1 (Daniel) y alexpm73435 (Alex). La siguiente tabla selecciona commits legibles en las capturas; no pretende sustituir la exportación completa del historial.

6.2 AVANCES Y COMMITS DE LA SEGUNDA ETAPA

Fecha	Autor	Cambio y utilidad
7 oct.	Sofía	Botón UNO con temporizador y representación de cartas rivales.
10 oct.	Daniel	Integración de API en la interfaz.
28 oct.	Alex	Programa de prueba del servidor.
30 oct.	Daniel	Cliente de prueba desarrollado en Python.
2 nov.	Alex	Incorporación de multijugador por sockets TCP.
4 nov.	Daniel	Correcciones del menú y cambios en Python.
4 nov.	Ángel	Corrección de reutilización de nombres en diferentes partidas.
4 nov.	Ángel	Visibilidad del botón de salir antes de unirse a sala.
4 nov.	Ángel	Registro de jugadores que participan en cada partida.
4 nov.	Ángel	Opción de revancha después del ganador, controlada por el creador.

Commits on Oct 30, 2026

Cliente python
danielrmzkuhn1 committed last week — 184e0eb

Commits on Oct 28, 2026

programa prueba servidor
alexpm73435 committed last week — 8fc21e5

Commits on Oct 10, 2026

Api en interfaz
danielrmzkuhn1 committed last month — 4027ade

Commits on Oct 7, 2026

Merge pull request #6 from danielrmzkuhn1/interfaz-mesa
sofii15 authored on Oct 7 — Verified — 355b0fa

Boton UNO con temporizador, mesa con cartas rivales
sofii15 committed on Oct 7 — 1d6d6f9

Commits on Nov 2, 2026

agregar miltijugador por sockets TCP
alexpm73435 committed 4 days ago — c215531

agregar miltijugador por sockets TCP
alexpm73435 committed 4 days ago — 6a22e6b

Commits on Nov 4, 2026

Add creator-controlled rematches
angelddev committed 2 days ago — 23d859e

Track players participating in each match
angelddev committed 2 days ago — a69714a

Merge branch 'DB_fix'
angelddev committed 2 days ago — e626634

Hide leave button before joining room
angelddev committed 2 days ago — a22f1ba

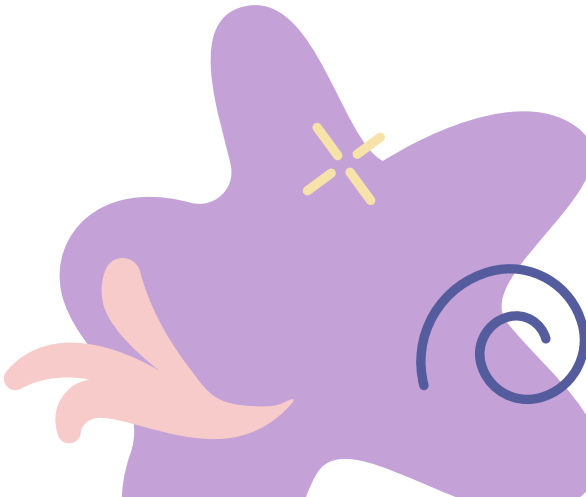
Fix player name reuse across matches
angelddev committed 2 days ago — 57913bc

Menu casi final
danielrmzkuhn1 committed 2 days ago — acf0361

Se corrigio en python
danielrmzkuhn1 committed 2 days ago — 5e292e6

Correccion Menu
danielrmzkuhn1 committed 2 days ago — 35b19ab





6.3 Integración y actualización entre equipos

Cada integrante pudo actualizar su copia del proyecto con git pull para obtener los commits más recientes. En paralelo con el código, cada instalación local de MySQL necesitó incorporar la nueva tabla partida_jugadores. La actualización de código por sí sola no modifica automáticamente los esquemas de las bases locales; por ello fue necesario ejecutar la sentencia CREATE TABLE antes de probar la nueva versión.

6.4 Importancia del historial de integración

Evidencía que el funcionamiento del sistema dependió de coordinar cambios transversales entre módulos y de aplicar ajustes derivados de las pruebas de uso. Dado que una nueva acción requiere sincronización simultánea entre la interfaz en C#, la transmisión por sockets, el servidor y la base de datos, los commits reunidos en la rama principal reflejan tanto avances progresivos como correcciones posteriores a la implementación del multijugador (como la validación de nombres repetidos, la visibilidad de botones, el registro de participantes y la opción de nueva partida). Por ello, para futuras entregas se recomienda documentar en cada modificación el problema específico que resuelve y los archivos o módulos afectados.

7. Dificultades, decisiones y mejoras

Situación observada	Solución adoptada o acción necesaria
Partidas limitadas a una computadora.	Se separó el servidor Python y se implementó comunicación TCP para clientes C# en red Wi-Fi.
Riesgo de estados diferentes entre clientes.	El servidor mantiene la versión del estado y publica cambios; las solicitudes repetidas se reconocen por identificador.
Privacidad de cartas de los rivales.	La vista del motor incluye la mano del usuario autorizado y el número de cartas de los demás.
No existía lista explícita de jugadores por partida.	Se agregó partida_jugadores con PK compuesta y dos FK para registrar participantes al iniciar.
Un nombre podía volver a utilizarse en otra partida.	La operación obtener_jugador busca un nombre existente sin diferenciar mayúsculas y minúsculas y reutiliza su id.
Fin de partida y continuidad de la sala.	Se incorporó la revancha del creador, con un nuevo registro de partida y los mismos participantes.
Actualización de BD al obtener cambios del repositorio.	Cada instalación de MySQL debe ejecutar las migraciones; falta anexar el DDL exacto de salas_wifi al script de entrega.

7.1 ASPECTOS TÉCNICOS QUE EXIGIERON MAYOR INTEGRACIÓN

La transición del juego a una red local presentó tres desafíos técnicos principales: sincronizar el estado de la partida, gestionar las identidades de los usuarios y adaptar la base de datos. Para evitar desajustes por fallos o retrasos en la conexión, se centralizó toda la lógica y la validación de reglas en el servidor (motor.py) utilizando control de versiones por solicitud, dejando al cliente en C# únicamente la tarea de representar los datos aprobados. Por otro lado, para evitar nombres duplicados en una misma sala sin impedir que un usuario juegue en distintas partidas, se separó el historial persistente (consultado mediante obtener_jugador sin distinguir mayúsculas) de la validación temporal de la sala activa (validar_entrada). Finalmente, para registrar con precisión a todos los participantes —incluso a quienes no realizan movimientos—, se incorporó la tabla partida_jugadores con una relación muchos a muchos y clave primaria compuesta, lo que también exigió mantener sincronizados los cambios de esquema MySQL entre todos los entornos de prueba.

8. RESULTADOS, PRUEBAS Y CONCLUSIONES

8.1 FUNCIONALIDADES ALCANZADAS

Resultado	Evidencia que lo respalda
Juego UNO con reglas y turnos.	Implementaciones en Logica/Juego.cs y servidor/motor.py; pruebas unitarias de motor.
Conexión multijugador.	ApiService.cs y servidor_sockets.py; captura de sala en Wi-Fi y confirmación del equipo.
Salas con identificador y responsable.	Funciones crear, entrar, cancelar y revancha en api.py; controles en Form1.cs.
Registro de participantes y movimientos.	Tablas partida_jugadores e historial_partida; inserciones y consultas en api.py.
Interfaz con menú, mesa y cartas.	Capturas del menú, mesa local y pantalla de sala multijugador.

El proyecto evolucionó de una interfaz y un motor orientados a un solo equipo hacia una arquitectura en la que varias computadoras comparten el estado de la partida. Esta separación permitió distinguir tres responsabilidades: el cliente presenta la información y recibe entradas, el servidor valida las reglas y la base conserva los registros de juego.
La mejora más importante del modelo SQL es partida_jugadores. Con ella, las victorias y derrotas pueden atribuirse a quienes participaron realmente, en lugar de inferir participantes únicamente a partir de sus movimientos. El sistema también conserva los eventos de la partida, lo que permite reconstruir la secuencia de acciones registrada.
La organización mediante GitHub documenta aportaciones de los cuatro integrantes durante ambas etapas. Las nuevas opciones para salir de una sala, mantener los nombres entre partidas y comenzar una revancha responden a situaciones que aparecen al utilizar el juego en red.

8.2 Validación mediante pruebas y revisión del código

El repositorio contiene tres archivos de pruebas dirigidos a partes diferentes del sistema. La separación ayuda a detectar si un error proviene de las reglas, de la persistencia o del transporte. En la revisión para este reporte se ejecutó el conjunto test_motor.py y sus 16 casos finalizaron correctamente. No se presenta ese resultado como una verificación integral de la instalación: las pruebas de la API y los sockets requieren dependencias y condiciones adicionales.

Conjunto	Qué se verifica	Evidencia disponible
Motor UNO	Número de cartas, reparto, turnos, cartas especiales, UNO, victoria y conservación de cartas.	16 pruebas unitarias ejecutadas satisfactoriamente durante la revisión.
Servicios y SQL	Autenticación, identidad, solicitudes repetidas, versiones, participantes y revancha.	Casos automatizados presentes en test_api.py; emplean simulación con SQLite.
Sockets TCP	Mensajes fragmentados o concatenados, publicaciones, privacidad, errores y plazos UNO.	Casos definidos en test_sockets.py; requieren ejecutar el servidor de pruebas.
Uso entre computadoras	Conexión a una misma sala desde varios clientes, reparto y jugadas visibles en la red.	Funcionamiento comunicado por el equipo; falta agregar capturas simultáneas de la partida.

ANEXO A : SCRIPT SQL DEL MODELO RELACIONAL

Script correspondiente a la carpeta base_datos del proyecto. Incluye cuatro tablas y sus llaves; la tabla auxiliar de salas se administra en el entorno MySQL del equipo y debe incorporarse al archivo de entrega cuando se disponga de su definición.

```
CREATE DATABASE uno_db;
USE uno_db;

CREATE TABLE jugadores (
  id_jugador INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL
);

CREATE TABLE partidas (
  id_partida INT AUTO_INCREMENT PRIMARY KEY,
  fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
  id_ganador INT,

  FOREIGN KEY (id_ganador)
    REFERENCES jugadores(id_jugador)
);

CREATE TABLE partida_jugadores (
  id_partida INT NOT NULL,
  id_jugador INT NOT NULL,
  PRIMARY KEY (id_partida, id_jugador),
  FOREIGN KEY (id_partida)
    REFERENCES partidas(id_partida),
  FOREIGN KEY (id_jugador)
    REFERENCES jugadores(id_jugador)
);

CREATE TABLE historial_partida (
  id_evento INT AUTO_INCREMENT PRIMARY KEY,
  id_partida INT NOT NULL,
  id_jugador INT NOT NULL,
  turno INT NOT NULL,
  accion VARCHAR(30) NOT NULL,
  color_carta VARCHAR(20),
  valor_carta VARCHAR(20),

  FOREIGN KEY (id_partida)
    REFERENCES partidas(id_partida),

  FOREIGN KEY (id_jugador)
    REFERENCES jugadores(id_jugador)
);
```



ANEXO B : EVIDENCIA DE INTERFAZ

Las figuras distinguen las pantallas de la primera etapa de la pantalla correspondiente a la sala TCP, de forma que los cambios de alcance sean claros.







En esta versión se observa una sala identificada por código y un cliente conectado al servidor TCP. La pantalla permite reunir participantes antes de que el creador inicie la partida. El campo de IP:puerto distingue la ejecución en red de la primera interfaz local.

ANEXO C: PRUEBAS Y FRAGMENTOS DE IMPLEMENTACIÓN

C.1 Registro SQL de participantes

La siguiente función se ejecuta cuando el creador inicia una partida, después de preparar el estado del juego.

```
def registrar_participantes(cur, id_partida, jugadores):
    for jugador in jugadores:
        cur.execute(
            'INSERT INTO partida_jugadores(id_partida,id_jugador) VALUES (%s,%s)',
            (id_partida, jugador['id'])
        )
```

C.2 Consulta ilustrativa para resultados individuales

Esta consulta es una propuesta derivada del esquema relacional; no se presenta como una consulta ya integrada en el código del juego.

```
SELECT p.id_partida, p.fecha, j.nombre,
    CASE WHEN p.id_ganador = j.id_jugador
        THEN 'Victoria' ELSE 'Derrota' END AS resultado
FROM partida_jugadores AS pj
JOIN jugadores AS j ON j.id_jugador = pj.id_jugador
JOIN partidas AS p ON p.id_partida = pj.id_partida
WHERE p.id_ganador IS NOT NULL
ORDER BY p.fecha, p.id_partida, j.nombre;
```

C.3 Escenarios cubiertos por pruebas Python

Archivo de pruebas	Casos incluidos en el repositorio
test_motor.py	Reparto de 108 cartas, turnos, saltos, reversa, castigos, UNO, ganador y conservación del mazo.
test_api.py	Autenticación, versiones obsoletas, reintentos, nombres repetidos, participantes, revancha y persistencia simulada.
test_sockets.py	Notificaciones TCP, privacidad de manos, mensajes fragmentados, expiración UNO, control de revancha y errores.

Las pruebas de api.py utilizan SQLite en memoria como sustituto local para validar operaciones, no una conexión real a MySQL. Para verificar completamente la instalación, deben complementarse con pruebas del servidor y de la base MySQL utilizada por el equipo.

ANEXO D : PROCEDIMIENTO DE INTEGRACIÓN Y VERIFICACIÓN

D.1 Preparación de las computadoras

El siguiente procedimiento resume lo que se necesita para reproducir una prueba en una red local. Se deriva de los archivos de configuración y de las clases de conexión revisadas; algunos pasos dependen de la instalación del laboratorio y deben completarse con la configuración real del equipo.

1. Actualizar los archivos del repositorio. Verificar que estén disponibles el proyecto de Windows Forms y los módulos Python del directorio servidor.
2. En el equipo anfitrión, iniciar MySQL y confirmar la existencia de uno_db, incluidos jugadores, partidas, historial_partida, partida_jugadores y salas_wifi.
3. Preparar el entorno de Python con las dependencias que importan los módulos del servidor, entre ellas mysql.connector y pydantic. Configurar la contraseña de MySQL mediante UNO_DB_PASSWORD.
4. Iniciar servidor_sockets.py en el equipo anfitrión. Por defecto escucha en el puerto TCP 8000 y permite modificarlo mediante UNO_PORT.
5. Conectar las computadoras cliente a la misma red local. En el menú de C#, reemplazar 127.0.0.1 por la IP local del anfitrión y usar el mismo puerto.
6. Crear una sala desde una computadora, compartir el código, unir al menos otro cliente, iniciar y verificar que las jugadas actualicen ambos estados.
7. Si el cliente no puede conectarse, revisar la dirección IP, el puerto, el firewall y que el servidor indique que MySQL está disponible. El código comprueba la tabla salas_wifi cuando atiende la operación de salud; por lo tanto, esta tabla debe existir aunque su CREATE TABLE todavía esté pendiente de incorporarse al script versionado.

D.2 Consultas útiles para las evidencias de SQL

Estas sentencias permiten documentar una instalación real después de jugar una partida. Las dos primeras identifican las tablas disponibles; las siguientes muestran registros del juego. La salida dependerá de los datos guardados en MySQL.

```
USE uno_db;
SHOW TABLES;
SHOW CREATE TABLE salas_wifi;

SELECT id_partida, fecha, id_ganador
FROM partidas ORDER BY id_partida DESC;

SELECT id_partida, id_jugador
FROM partida_jugadores
ORDER BY id_partida DESC, id_jugador;

SELECT id_partida, turno, id_jugador, accion,
    color_carta, valor_carta
FROM historial_partida
ORDER BY id_partida DESC, turno;
```

